

TECHNICAL WHITE PAPER

PhishGuard

An Open-Source Phishing Detection & Awareness Tool

Protecting Individuals and Organisations from Email-Based Threats

Author

Ekeh Nzubechukwu Favour

Junior Cybersecurity Analyst

BSc. Cyber Security

Document Type	Technical White Paper Portfolio Project Proposal
Project Name	PhishGuard Phishing Detection & Awareness Tool
Version	1.0 March 2026
Status	Active Development MVP Phase
Domain	Cybersecurity / Defensive Security / Web Application
Target Audience	Cybersecurity Professionals, SMEs, Academic Institutions
Contact	nzubeak@gmail.com

CONFIDENTIAL FOR PORTFOLIO AND PROFESSIONAL REVIEW PURPOSES

EXECUTIVE SUMMARY

Key Objectives

Deliver a functional MVP within five weeks. Demonstrate proficiency in Python, Flask, REST API integration, threat intelligence correlation, and secure software development practices. Deploy publicly to establish a verifiable, portfolio-grade cybersecurity project.

80%+

of breaches start with phishing

7

detection signals analysed

5 weeks

MVP build timeline

Free

open-source & publicly deployed

1. PROBLEM STATEMENT

1.1 The Global Phishing Epidemic

Email-based phishing attacks have grown in both sophistication and volume over the past decade. According to the Anti-Phishing Working Group (APWG), phishing attacks reached record levels in 2024, with millions of unique phishing sites detected monthly. Attackers use social engineering, brand impersonation, and increasingly AI-generated content to deceive victims into revealing credentials, transferring funds, or installing malware.

Traditional defences spam filters, blocklists, and signature-based detection are reactive and fail to catch novel, zero-day phishing campaigns. End users remain the last line of defence, yet most lack the tools or training to analyse suspicious communications independently.

1.2 The Nigerian & African Context

Nigeria's rapidly expanding digital economy driven by fintech, e-commerce, and mobile banking adoption has made it a significant target for phishing campaigns. Business Email Compromise (BEC) attacks targeting Nigerian organisations have increased substantially, with losses running into billions of naira annually. Despite this, the vast majority of small and medium enterprises (SMEs) operate without dedicated security tools or IT security personnel.

Academic institutions, which are training the next generation of professionals, similarly lack practical cybersecurity awareness infrastructure. There is a demonstrable market gap for lightweight, free, and locally contextualised phishing detection tools.

1.3 Gap in Existing Solutions

Tool / Approach	Cost	Accessibility	Limitation
Enterprise SIEM platforms	High (\$\$\$\$)	IT teams only	Overkill for individuals/SMEs
Google Safe Browsing	Free	Browser only	No email header analysis
Manual analysis	Free	Experts only	Requires deep technical knowledge
VirusTotal (direct)	Free	Technical users	No scoring, no plain-English output
PhishGuard (proposed)	Free	Anyone	Comprehensive, guided verdict

2. PROPOSED SOLUTION PHISHGUARD

2.1 Solution Overview

PhishGuard is a web-based email threat analysis platform. A user pastes a raw email or suspicious URL into the interface and receives, within seconds, a structured risk assessment that identifies exactly which threat signals were triggered and why the verdict was reached. The system is designed to be understandable by non-technical users while remaining rigorous enough for use by security-aware professionals.

2.2 Core Features MVP Scope

- Email header extraction and anomaly detection (From/Reply-To mismatch, SPF/DKIM failure)
- Automated URL extraction from email body using HTML parsing
- Redirect chain unwinding to expose concealed final destinations
- VirusTotal API integration for multi-engine URL and domain threat scoring
- WHOIS domain age lookup to identify newly registered phishing domains
- Urgency keyword detection in subject and body text
- Weighted risk scoring engine producing a 0–14 point verdict
- Plain-English verdict output with per-signal explanations
- Flask web interface accessible via standard browser
- Deployment on Render.com with public URL

2.3 Design Principles

Simplicity first. The tool must be usable by a non-technical office worker with no security background. Verdicts are expressed in plain language, not security jargon.

Transparency. Every point in the risk score is individually explained. Users understand precisely what triggered the verdict there are no black-box results.

Security by design. API keys are stored as environment variables. No email content is logged or persisted. The tool processes data transiently and returns results only to the requesting session.

Open source and extensible. The full codebase is published on GitHub under an open-source licence, enabling the security community to review, extend, and improve the tool.

3. SYSTEM ARCHITECTURE & TECHNICAL DESIGN

3.1 High-Level Architecture

PhishGuard follows a classic three-tier web application architecture with a dedicated threat intelligence integration layer:

Layer	Components	Responsibility
Presentation	Flask + Jinja2 + HTML/CSS	Accepts user input, renders structured results and verdict UI
Application	Python parser.py, intel.py, scorer.py, app.py	Email parsing, URL extraction, scoring logic, request routing
Intel Layer	VirusTotal API, python-whois	External threat intelligence lookup and domain verification
Infrastructure	Render.com (cloud), GitHub (source)	Hosting, CI/CD, version control, public access

3.2 Data Flow

The following sequence describes a complete analysis request from submission to verdict:

1. User submits raw email text or URL via the web interface (HTTP POST to Flask route).
2. parser.py extracts From, Reply-To, Return-Path, Subject headers and all body URLs.
3. Header analyser checks for domain mismatches and attempts SPF/DKIM record lookup via DNS.
4. URL extractor (BeautifulSoup) identifies all hyperlinks; requests library follows redirect chains.

5. intel.py submits each URL and domain to VirusTotal API; retrieves engine vote count.
6. python-whois fetches domain registration date; age is calculated in days.
7. scorer.py applies the weighted signal table and produces a total score and verdict string.
8. Result object is passed to Jinja2 template; user receives colour-coded verdict page.

3.3 Module Structure

The project follows a clean separation of concerns across four core Python modules:

Component	Technology	Purpose
parser.py	email (stdlib)	Parses raw .eml input; extracts all headers and body content
intel.py	requests, python-whois	Calls VirusTotal API; runs WHOIS domain age checks
scorer.py	Python logic	Applies 7-signal weighted scoring; returns verdict and explanations
app.py	Flask, python-dotenv	HTTP routing, environment config, template rendering
templates/	Jinja2 HTML	index.html (form), result.html (verdict display)
.env file	python-dotenv	Securely stores VirusTotal API key outside source code

4. RISK SCORING METHODOLOGY

4.1 Scoring Philosophy

The PhishGuard risk engine uses a weighted additive scoring model. Each detectable threat signal contributes a fixed number of points to the total. The final score is mapped to one of three verdict tiers. This approach is intentionally transparent every point is traceable to a specific, explainable detection rule.

Design Rationale

Weighted additive models are widely used in real-world threat scoring systems (e.g. SpamAssassin, Google Safe Browsing). They are interpretable, auditable, and easy to tune critical properties for a tool intended to explain its reasoning to non-expert users.

4.2 Signal Definitions & Point Values

Signal	Points	Rationale
From / Reply-To domain mismatch	+2 pts	Legitimate senders rarely route replies to a different domain than the From address

Signal	Points	Rationale
SPF or DKIM record failure	+2 pts	Authentication headers are foundational email security standards; failure is a strong phishing indicator
VirusTotal URL flagged (3+ engines)	+4 pts	Consensus across multiple AV engines is a high-confidence malicious signal
Sender domain registered < 30 days	+2 pts	Phishing infrastructure is almost always newly registered; old domains cost more to abuse
URL uses raw IP address	+1 pt	Legitimate services virtually never embed IP addresses in email links
Redirect chain of 3+ hops	+1 pt	Multi-hop redirects are commonly used to evade simple blocklist checks
High-urgency keywords detected	+1 pt	Social engineering relies on urgency; words like 'verify now', 'suspended', 'immediate action' are strong indicators
Mismatched display text vs actual URL	+1 pt	Hyperlinks that display a trusted domain but point elsewhere are a classic deception technique

4.3 Verdict Thresholds

Score Range	Verdict	Recommended Action
0 – 2	SAFE	No action required. Email passes all checks.
3 – 6	SUSPICIOUS	Exercise caution. Do not click links. Verify sender through a separate channel.
7 – 14	LIKELY PHISHING	Do not interact. Report to your IT or security team. Delete immediately.

5. TECHNOLOGY STACK & JUSTIFICATION

All tools selected for PhishGuard are free, open-source, and widely used in production cybersecurity environments. This ensures that skills developed during this project are directly transferable to professional roles.

Component	Technology	Purpose
Language	Python 3.10+	Industry-standard for security tooling; extensive library ecosystem; strong job market demand
Web Framework	Flask 3.x	Lightweight and production-ready; minimal overhead; standard for Python security tools
Email Parsing	email (stdlib)	Built-in Python library; handles all MIME types natively; no third-party dependency
HTTP Client	requests 2.x	De facto standard Python HTTP library; handles redirects, sessions, and timeout control
HTML Parsing	BeautifulSoup4	Robust link extraction from email HTML bodies; handles malformed markup gracefully
Domain Intel	python-whois	WHOIS registration date lookup; single function call returns structured domain data
Threat Intel	VirusTotal API v3	Free tier; 70+ AV engines; RESTful JSON API; industry-standard threat intelligence source
Config Security	python-dotenv	Loads API keys from .env file; prevents credential exposure in source code
Deployment	Render.com	Free tier; GitHub integration; automatic deploys; HTTPS by default
Version Control	Git + GitHub	Standard source control; enables public portfolio visibility and collaborative review

Security Note API Key Management

The VirusTotal API key will be stored exclusively in a .env file, loaded at runtime via python-dotenv. The .env file is listed in .gitignore and will never appear in the public GitHub repository. On Render.com, the key is set as an environment variable in the dashboard not in any configuration file.

6. DEVELOPMENT ROADMAP

6.1 Five-Week MVP Plan

Week 1 Foundation & Email Parser

- Configure Python virtual environment and install all dependencies
- Implement parser.py: raw .eml input → structured header dictionary + body URL list
- Test parser against 5 real spam emails saved from inbox
- Set up Git repository with initial commit and .gitignore

Week 2 Threat Intelligence Integration

- Implement intel.py: VirusTotal URL scan function with rate limiting (4 req/min)
- Implement WHOIS domain age function in intel.py

- Handle API error states: timeouts, rate limit 429 responses, invalid URLs
- Test against 5 known phishing URLs from PhishTank.com and 5 clean URLs

Week 3 Risk Scoring Engine

- Implement scorer.py with all 8 weighted signals
- Implement get_explanation() function returning per-signal plain-English text
- Write unit tests for 10+ scoring input combinations
- Conduct end-to-end pipeline test on 10 sample emails

Week 4 Flask Web Interface

- Implement app.py with POST route accepting email text and URL inputs
- Build index.html: clean input form with textarea and submit button
- Build result.html: colour-coded verdict badge, score breakdown, per-signal cards
- Integrate python-dotenv; test full flow at localhost:5000

Week 5 Polish, Deploy & Document

- Deploy to Render.com; configure environment variables; verify HTTPS
- Test live deployment with 5 phishing samples; capture screenshots
- Write professional GitHub README with setup guide and screenshots
- Publish LinkedIn article documenting the build process and findings
- Add live URL and GitHub link to CV

6.2 Post-MVP Enhancement Roadmap

Following the MVP launch, the following features are planned for subsequent development sprints:

- Attachment analysis: detect malicious macros and embedded executable code in email attachments
- Machine learning classifier: train a Naive Bayes or Random Forest model on a labelled phishing dataset
- Batch analysis: allow upload and processing of multiple .eml files simultaneously
- Report export: generate a downloadable PDF analysis report for sharing with IT teams
- Nigerian threat context: integrate local threat feeds and BEC pattern detection specific to Nigerian financial sector targets
- REST API: expose analysis endpoints for integration with third-party tools and SIEM platforms

7. PORTFOLIO & CAREER IMPACT

7.1 Skills Demonstrated

Completion of PhishGuard will provide verifiable, documented evidence of the following professional competencies:

Skill Area	Evidence Produced
Python Development	4-module application with clean separation of concerns and documented functions
REST API Integration	VirusTotal API calls with rate limiting, error handling, and JSON parsing
Threat Intelligence	Multi-signal analysis combining WHOIS, DNS, and AV engine data
Secure Coding	Environment variable key management, no data persistence, input sanitisation
Web Application Dev	Flask backend with Jinja2 templating deployed to production
Technical Writing	White paper, README, LinkedIn article all publicly accessible
Security Mindset	Designed a detection system from first principles using OWASP and NIST concepts

7.2 Target Job Roles

The skills and artefacts produced by PhishGuard are directly relevant to the following entry-level and junior cybersecurity positions:

- SOC Analyst (Tier 1 & 2) email triage, threat analysis, and alert investigation
- Junior Penetration Tester understanding of phishing attack vectors and detection evasion
- Security Engineer Python scripting, API integration, and security tool development
- Cybersecurity Consultant client-facing threat education and awareness tool delivery
- Threat Intelligence Analyst threat feed integration, indicator analysis, and scoring methodology

Nigerian Market Relevance

Nigeria's Central Bank (CBN), major commercial banks (GTBank, Zenith, Access), telecoms (MTN, Airtel), and fast-growing fintechs (Flutterwave, Paystack, Kuda) all have active cybersecurity hiring programmes. Demonstrating a working phishing detection tool speaks directly to the threats these organisations face daily.

8. CONCLUSION

Phishing is not a technical problem that will be solved by enterprise tools alone it requires accessible, transparent, and community-driven solutions that empower ordinary users to protect themselves. PhishGuard is a contribution to that effort, built by a junior analyst who understands both the technical mechanisms of email-based attacks and the human realities of the organisations they target.

This white paper has documented a rigorous, methodical approach to building a security tool from first principles: starting with a clearly defined problem, designing a transparent detection architecture, selecting a justified technology stack, and committing to a realistic, time-bounded delivery plan.

Upon completion of the MVP, PhishGuard will be a publicly accessible, open-source tool with a live deployment, a documented codebase, and a published technical write-up providing concrete, verifiable evidence of applied cybersecurity competence to any prospective employer, collaborator, or academic reviewer.

Ekeh Nzubechukwu Favour

Junior Cybersecurity Analyst |
| Nzubeak@email.com | <https://github.com/nzubeak> |
www.linkedin.com/in/nzubechukwu-ekkeh-60575b236
Abuja, Nigeria | March 2026

This document was prepared as part of a structured cybersecurity portfolio development programme.